STUDY AT YOUR OWN RISK

1.) Remote User Authentication Principles (Easy – Exam Oriented)

User authentication means verifying that a user is really who he/she claims to be before allowing access to a system.

It has two main steps:

1. **Identification**
   – User provides an identity (User ID, username, email, etc.)
2. **Verification**
   – User proves the identity using authentication information (password, key, biometric, etc.)

NIST Model for Electronic User Authentication

NIST defines electronic authentication as the process of **establishing confidence in a user's identity over a network**.

Main entities:

- **Subscriber (User)** – Person who wants access
- **Registration Authority (RA)** – Verifies user identity
- **Credential Service Provider (CSP)** – Issues credentials (password, token, certificate)
- **Verifier** – Checks the credentials
- **Relying Party (RP)** – Grants access based on verification

Two steps:

1. **Identification** – User gives ID
2. **Verification** – User proves ID using password, token, biometric, etc.

Purpose of Remote User Authentication

When users access systems over a **network or Internet**, authentication is needed to:

- Prevent unauthorized access
- Support access control
- Ensure accountability

Means of Authentication (From Textbook)

1. **Something the user knows**
   – Password, PIN, secret answer
2. **Something the user has**
   – Smart card, token, cryptographic key
3. **Something the user is (static biometrics)**
   – Fingerprint, face, iris
4. **Something the user does (dynamic biometrics)**
   – Voice, handwriting, typing pattern

Mutual Authentication

Both sides verify each other:

- Client proves identity to server
- Server proves identity to client

This prevents:

- Masquerading (pretending to be someone else)
- Replay attacks
- Fake servers

One-Way Authentication

Only **sender is authenticated** to receiver.

Used in:

- Email
- Digital signatures

Receiver verifies:

- Message is from the claimed sender
- Message is not modified

But sender does not verify receiver.

Protection Against Replay Attacks

To ensure message **freshness**, two methods are used:

1. **Timestamps**
   – Accept message only if time is within valid window
2. **Challenge–Response (Nonce)**
   – Server sends a random number (nonce)
   – Client returns it encrypted to prove freshness

One-Line Exam Answer

Remote user authentication is the process of verifying a user's identity over a network using identification and verification mechanisms such as passwords, cryptographic keys, and challenge–response or timestamp techniques to prevent impersonation and replay attacks.

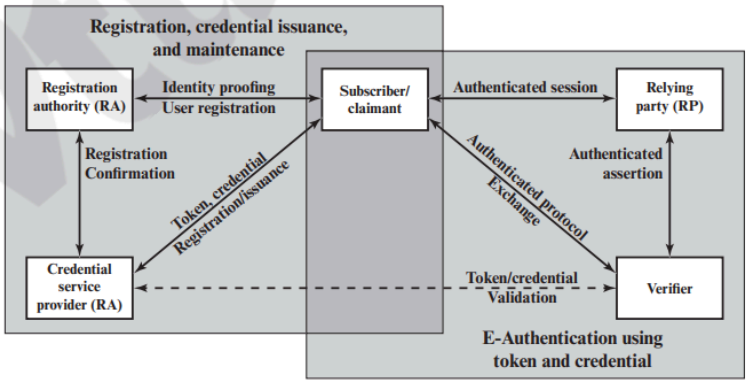

Figure 15.1 The NIST SP 800-63-2 E-Authentication Architectural Model

2) Kerberos (Easy – Exam Oriented)

Kerberos is a **trusted third-party authentication system** used to securely authenticate users and servers in a network using **symmetric key cryptography**.

Kerberos (Complete Exam-Oriented Answer)

---

1) Motivation for Kerberos

Kerberos was developed at **MIT (Project Athena)** to solve problems in network authentication:

Problems in networks:

- Passwords can be **eavesdropped**.
- Servers can be **impersonated**.
- Replay attacks can occur.
- Users must authenticate to **many servers**.

Kerberos provides:

- **Secure authentication**
- **Mutual authentication**
- **Single Sign-On**
- No password sent in plain text.
- Uses **symmetric key cryptography** and a **trusted third party (KDC)**.

---

2) Kerberos Version 4

Kerberos V4 is a **ticket-based authentication system**.

Main Components

- **Client (C)**
- **Authentication Server (AS)**
- **Ticket Granting Server (TGS)**
- **Application Server (V)**
- **Key Distribution Center (KDC = AS + TGS)**

Basic Working (V4)

1. Client → AS : Requests TGT
2. AS → Client : Sends
   - Session key $K_{c,tgs}$
   - Ticket Granting Ticket (TGT)
3. Client → TGS : Sends TGT + Authenticator
4. TGS → Client : Sends Service Ticket $Ticket_v$
5. Client → Server V : Sends Ticket_v + Authenticator
6. Server → Client : Optional mutual authentication reply

Uses:

- **DES**
- **Timestamps**
- **Tickets**
- **Session keys**

---

3) Kerberos Version 5

Kerberos V5 is an improved and more flexible version.

New Features in V5

1. **Supports multiple encryption algorithms** (not only DES)
2. **Supports different network addresses** (not only IP)
3. **Better ticket lifetime control** (start time, end time, renew time)
4. **Authentication forwarding**
5. **Inter-realm authentication with fewer trust links**
6. Uses **ASN.1 encoding** for messages
7. Supports:
    - Forwardable tickets
    - Renewable tickets
    - Proxy tickets
    - Postdated tickets

Adds fields:

- Realm
- Options
- Nonce
- Sub-key
- Sequence number

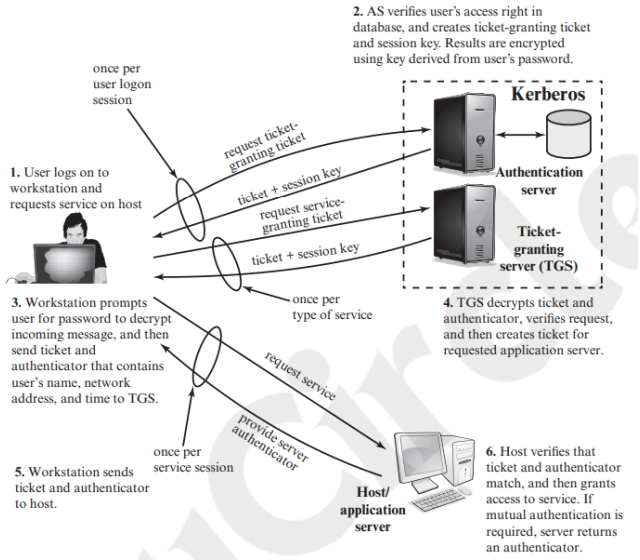

Figure 15.2 Overview of Kerberos

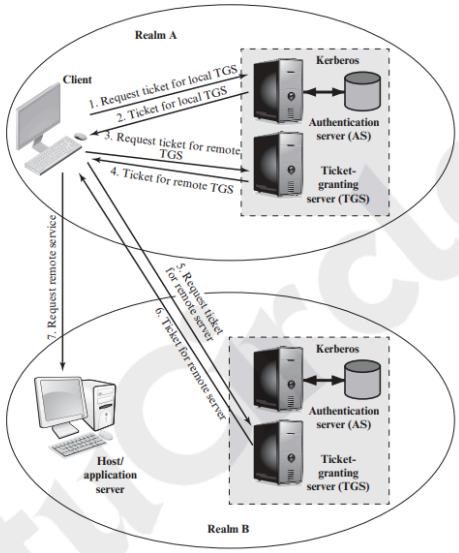

Figure 15.4 Request for Service in Another Realm

---

3) Differences Between Kerberos V4 and V5

Feature	Version 4	Version 5
Encryption	Only DES	Any algorithm
Network type	Only IP	Any address type
Message format	Ad-hoc	ASN.1 / BER

Ticket lifetime	Max ~21 hours	Flexible start & end time
Authentication forwarding	Not supported	Supported
Inter-realm	N^2 trust links	Hierarchical trust
Replay protection	Timestamp only	Timestamp + nonce + sequence no
Ticket flags	Limited	Forwardable, renewable, proxy, etc.
PCBC mode	Used (weak)	Replaced
Double encryption	Present	Removed

One-Line Exam Summary

Kerberos is a trusted third-party, ticket-based authentication protocol using symmetric encryption that provides secure mutual authentication. Version 5 improves upon Version 4 by supporting multiple encryption algorithms, flexible ticket lifetimes, inter-realm authentication, forwarding, and stronger replay protection.

4) Remote User Authentication using Asymmetric Encryption (Easy – Exam Oriented)

In this method, **public key cryptography** is used to authenticate a user over a network.

Each user has:

- **Public Key (PU)** – known to everyone
- **Private Key (PR)** – kept secret

Authentication is done using **challenge–response** and **digital signatures**.

Basic Principle

1. User A wants to prove identity to Server B.
2. B sends a random number (nonce) to A.
3. A encrypts/signs the nonce using **A's private key**.
4. A sends it back to B.
5. B decrypts it using **A's public key**.
6. If correct, A is authenticated.

This proves that A owns the private key → hence identity is verified.

Mutual Authentication

Both user and server authenticate each other:

1. A proves identity to B using private key.
2. B proves identity to A using its private key.
3. Both exchange nonces and verify.

This prevents:

- Masquerading
- Replay attacks
- Fake servers

Advantages

- No shared secret key needed in advance.
- Password is not transmitted.
- Secure against eavesdropping.
- Supports digital signatures and certificates.

One-Way Authentication using Asymmetric Encryption

Used in:

- Email security
- Digital signatures

Sender signs message with private key.
Receiver verifies using sender's public key.

One-line Exam Answer

Remote user authentication using asymmetric encryption verifies a user's identity by using public–private key pairs and challenge–response or digital signatures, ensuring secure authentication without transmitting secret keys over the network.

5) Web Security Considerations (Easy – Exam Oriented)

Web security means protecting web applications, servers, and users from attacks and unauthorized access.

Main security goals:

- **Confidentiality** – Data should be secret.
- **Integrity** – Data should not be altered.
- **Authentication** – Verify user identity.
- **Authorization** – Allow only permitted access.
- **Availability** – Services should be accessible.

Important Web Security Issues

1. **User Authentication**
   - Passwords, OTP, certificates, biometrics.
   - Prevent impersonation.
2. **Data Encryption**
   - Use SSL/TLS (HTTPS) to protect data in transit.
3. **Session Management**
   - Protect session IDs from hijacking.
   - Use secure cookies and timeouts.
4. **Access Control**
   - Ensure users access only allowed resources.
5. **Common Web Attacks**
   - **Phishing** – Fake websites stealing credentials.
   - **Man-in-the-Middle** – Attacker intercepts communication.
   - **Replay Attack** – Reusing captured authentication data.
   - **SQL Injection / XSS** – Injecting malicious code.
6. **Server and Application Security**
   - Regular updates and patches.
   - Firewalls and intrusion detection.

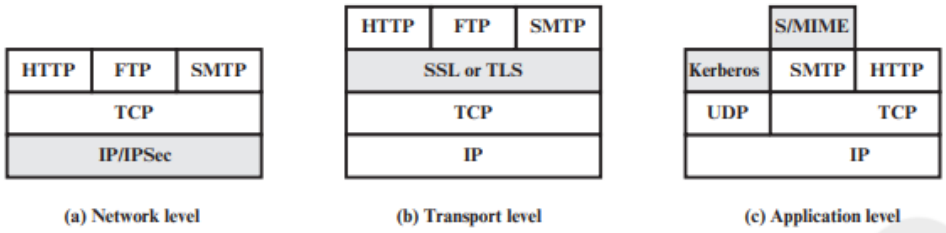

Figure 17.1 Relative Location of Security Facilities in the TCP/IP Protocol Stack

One-line Exam Answer

Web security considerations involve ensuring confidentiality, integrity, authentication, authorization, and availability of web resources using encryption, secure authentication, access control, and protection against common attacks like phishing, replay, and man-in-the-middle.

6) Transport Layer Security (TLS) – Complete Exam-Oriented Notes (as per Textbook)

---

1) Transport Layer Security (TLS)

TLS is a protocol that provides:

- **Confidentiality**
- **Integrity**
- **Authentication**

It secures communication between **client and server** (e.g., HTTPS).
TLS is the improved version of SSL.

How TLS Works (Simple)

1. Client connects to server.
2. Server sends its **digital certificate**.
3. Client verifies the certificate.
4. A **secret session key** is securely exchanged.
5. All further communication is encrypted using this session key.

TLS uses:

- Public key cryptography (for key exchange & authentication)
- Symmetric encryption (for data)
- MAC / Hash (for integrity)

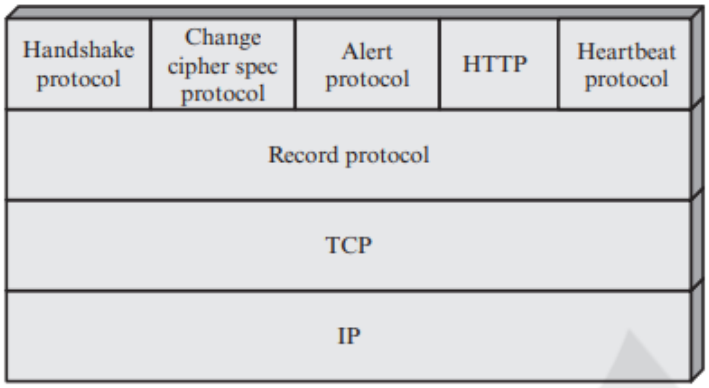

Figure 17.2 TLS Protocol Stack

---

2) TLS Record Protocol

This is the **lowest layer** of TLS.
All other TLS protocols (Handshake, Alert, Change Cipher Spec, Heartbeat) use it.

Functions of TLS Record Protocol

1. **Fragmentation**
   Breaks application data into small blocks.
2. **Compression (optional)**
   Compresses the data.
3. **MAC Generation**
   Adds Message Authentication Code for integrity.
4. **Encryption**
   Encrypts data using symmetric key.
5. **Encapsulation**
   Adds header and sends over TCP.

Services Provided

- **Confidentiality** (encryption)
- **Message Integrity** (MAC)

So, TLS Record Protocol:

Takes application data → adds security → sends securely.

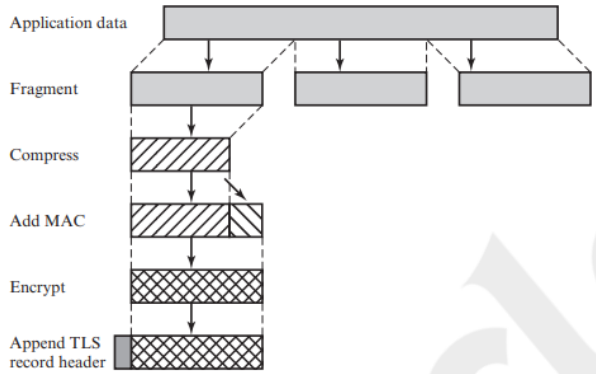

Figure 17.3 TLS Record Protocol Operation

---

3) Change Cipher Spec Protocol

This protocol is used to **notify a change in encryption parameters**.

It has only **one message**:

- Value = 1

Purpose

- Tells the other party:
  "From now on, use the newly negotiated keys and algorithms."

When it is used

After Handshake:

1. Keys and algorithms are agreed.
2. **ChangeCipherSpec message is sent.**
3. All further messages are encrypted with new keys.

Characteristics

- Very simple protocol.
- Uses TLS Record Protocol for transmission.
- Marks the transition from:
  - Unencrypted → Encrypted communication.

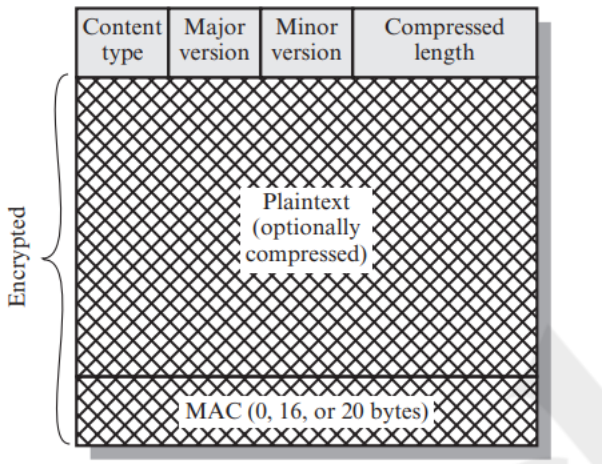

Figure 17.4 TLS Record Format

---

4) Alert Protocol

The **Alert Protocol** is used to send **error and warning messages** between client and server in TLS.

Each alert message has:

- **Level**:
  - *Warning* (connection may continue)
  - *Fatal* (connection must be closed)
- **Description**: Type of problem.

Common Alert Messages:

- `close_notify` – connection is closed normally
- `unexpected_message`
- `bad_record_mac`
- `handshake_failure`
- `bad_certificate`
- `certificate_expired`

If a **fatal alert** is sent, the TLS connection is immediately terminated.

Purpose:

- Inform errors
- Inform closure
- Maintain secure communication

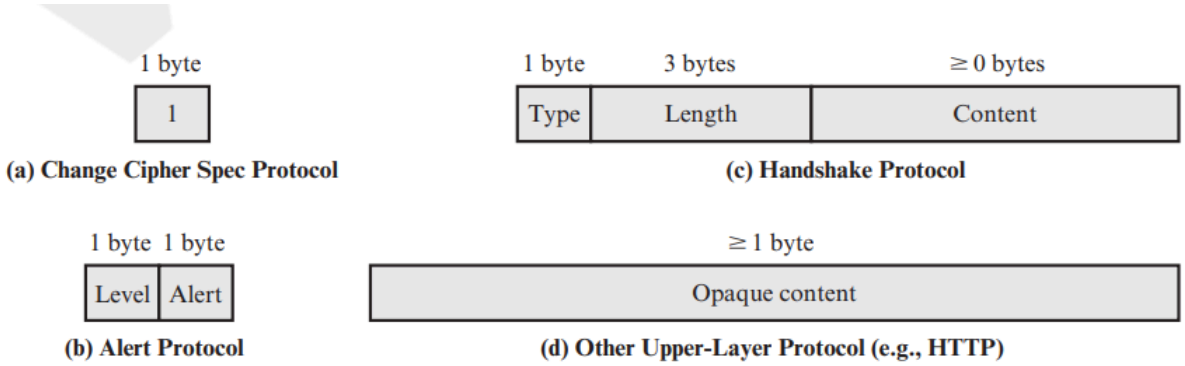

Figure 17.5 TLS Record Protocol Payload

---

5) Handshake Protocol

The **Handshake Protocol** is the most important part of TLS.
It is used to:

1. **Authenticate** the server (and optionally the client)
2. **Negotiate cryptographic algorithms**
3. **Exchange keys**
4. **Establish a secure session**

Main Steps

1. **ClientHello**
   - Sends supported cipher suites
   - Sends random number
2. **ServerHello**
   - Selects cipher suite
   - Sends its random number
   - Sends **digital certificate**
3. **Key Exchange**
   - Client generates pre-master secret
   - Encrypts it with server's public key
4. **ChangeCipherSpec**
   - Switch to encrypted communication
5. **Finished**
   - Both sides verify handshake integrity

After this, secure communication begins using symmetric session keys.

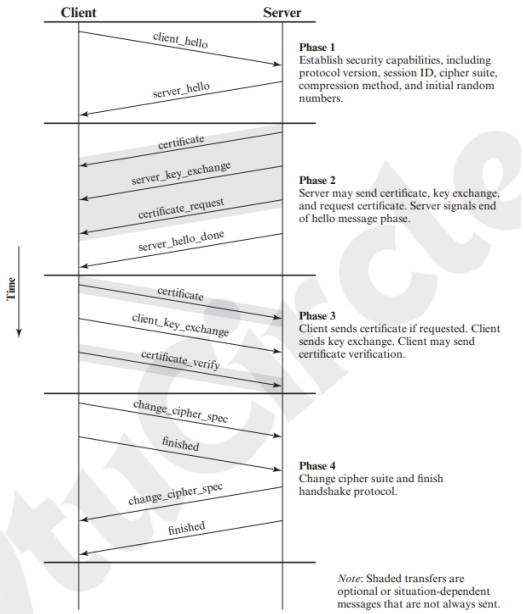

Figure 17.6 Handshake Protocol Action

---

6) Cryptographic Computations in TLS

TLS uses a combination of cryptographic algorithms to provide security:

1. **Key Exchange Algorithms**
   - RSA, Diffie–Hellman (DH), Elliptic Curve DH (ECDHE)
   - Used to securely generate a shared **pre-master secret**.
2. **Key Derivation**
   - From pre-master secret + client random + server random
   - A **master secret** is generated.
   - From master secret, the following are derived:
     - Encryption keys
     - MAC keys
     - Initialization vectors
3. **Encryption Algorithms (Symmetric)**
   - AES, 3DES, ChaCha20
   - Used for fast data encryption.
4. **Message Authentication**
   - HMAC with SHA-256 / SHA-384
   - Ensures data integrity and authentication.
5. **Hash Functions**
   - SHA family
   - Used in digital signatures and key generation.

So TLS combines:

- Public key cryptography (for key exchange & authentication)
- Symmetric encryption (for data)
- Hash & MAC (for integrity)

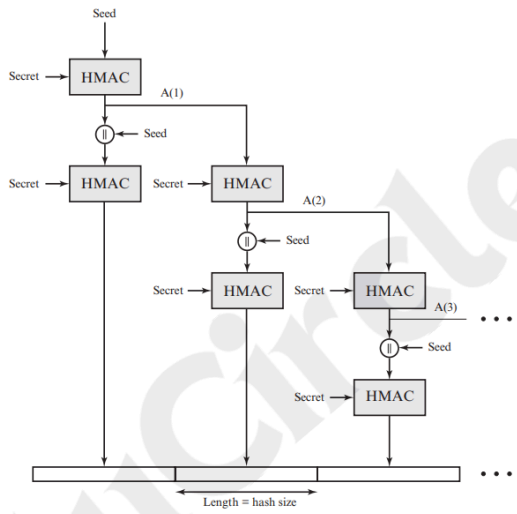

Figure 17.7 TLS Function `P_hash(secret, seed)`

---

7) Heartbeat Protocol

Heartbeat Protocol

The **Heartbeat Protocol** is used to:

- Check if the peer (client/server) is still alive.
- Keep long TLS connections active without renegotiation.

Working:

1. One side sends a **Heartbeat Request**.
2. Other side replies with a **Heartbeat Response** containing the same data.

It does **not** provide security by itself; it only uses the existing TLS secure channel.

Heartbleed Vulnerability

A serious flaw in the Heartbeat implementation (OpenSSL):

- Server sent more data than requested.
- Caused **memory leakage**.
- Attacker could read passwords, keys, and sensitive data.

SSL / TLS Attacks (Exam-Oriented)

Some important attacks discussed in TLS security:

1. **Man-in-the-Middle (MITM) Attack**
   Attacker intercepts communication between client and server and pretends to be both.
2. **Replay Attack**
   Old valid messages are captured and resent to gain unauthorized access.
3. **Downgrade Attack**
   Attacker forces client and server to use an older, weaker SSL/TLS version.
4. **BEAST Attack**
   Exploits weakness in CBC mode in TLS 1.0.
5. **CRIME / BREACH Attacks**
   Exploit compression in TLS to leak secret data.
6. **POODLE Attack**
   Targets SSLv3 padding weakness.
7. **Heartbleed Attack**
   Exploits a bug in the Heartbeat Protocol to read server memory.

TLS 1.3 (Latest Version)

TLS 1.3 is the latest and most secure version of TLS.

Main Improvements

1. **Simplified Handshake**
   - Faster (1-RTT, optional 0-RTT).
2. **Stronger Security**
   - Removes weak algorithms (RSA key transport, SHA-1, DES, RC4).
   - Only supports **forward secrecy** (ECDHE).
3. **Encrypted Handshake**
   - Most handshake messages are encrypted.
4. **No Downgrade Attacks**
   - Built-in protection.
5. **Fewer Cipher Suites**
   - Only strong and modern algorithms allowed.
6. **Improved Privacy**
   - Less information leaked to attackers.

---

7) Email Threats and Comprehensive Email Security (Easy – Exam Oriented)

---

1) Email Threats

Email is widely used, so it is a major target for attacks.

(a) Spoofing

Attacker sends email with fake sender address.
→ Receiver thinks it is from a trusted person.

(b) Phishing

Fake emails trick users into giving:

- Passwords
- Bank details
- OTPs

(c) Eavesdropping

Attacker reads email content during transmission if not encrypted.

(d) Message Modification

Attacker changes the content of the email.

(e) Replay Attack

Old valid email is captured and resent to cause confusion.

(f) Malware and Spam

Emails carry:

- Viruses
- Trojans
- Ransomware
  or大量 unwanted messages.

---

2) Comprehensive Email Security

To protect email, the following security services are required:

(a) Confidentiality

Only the intended receiver should read the email.
→ Achieved using **encryption** (e.g., PGP, S/MIME).

(b) Authentication

Verify the sender's identity.
→ Using **digital signatures**.

(c) Integrity

Ensure the message is not altered.
→ Using **hash functions + digital signature**.

(d) Non-Repudiation

Sender cannot deny sending the message.
→ Digital signatures provide proof.

(e) Access Control

Only authorized users can access mailboxes.

(f) Protection from Malware and Spam

→ Antivirus, spam filters, content scanning.

---

3) Technologies Used

(a) PGP (Pretty Good Privacy)

- Uses public key cryptography.
- Provides:
    - Encryption
    - Digital signature
    - Compression

(b) S/MIME (Secure MIME)

- Uses X.509 certificates.
- Provides:
    - Authentication
    - Encryption

    - Integrity
    - Non-repudiation

One-Line Exam Answer

Email threats include spoofing, phishing, eavesdropping, message modification, replay, and malware, and comprehensive email security is achieved using encryption, digital signatures, authentication, integrity checks, and secure email standards such as PGP and S/MIME.

8) S/MIME (Secure / Multipurpose Internet Mail Extensions)

S/MIME is a standard used to provide **security to e-mail** using **public key cryptography and X.509 certificates**.

Services Provided by S/MIME

1. **Confidentiality** – Message is encrypted.
2. **Authentication** – Sender is verified.
3. **Integrity** – Message is not changed.
4. **Non-repudiation** – Sender cannot deny sending the mail.

Working (Simple)

1. Sender encrypts the message using a **symmetric session key**.
2. Session key is encrypted with receiver's **public key**.
3. Sender signs the message using **private key**.
4. Receiver:
    - Decrypts session key using private key.
    - Verifies signature using sender's public key.

S/MIME uses:

- X.509 certificates
- RSA / AES / SHA

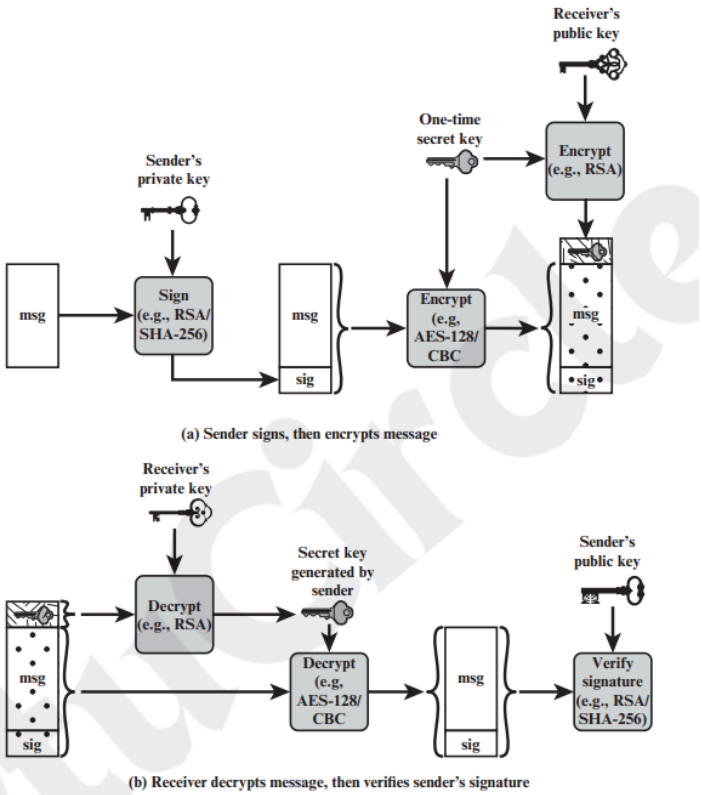

Figure 19.5 Simplified S/MIME Functional Flow

---

2) PGP (Pretty Good Privacy)

PGP is a popular software for **secure e-mail communication**.

Services Provided by PGP

1. **Confidentiality** – Encryption
2. **Authentication** – Digital signature
3. **Integrity** – Hash function
4. **Compression** – Reduces message size
5. **E-mail compatibility** – ASCII armor
6. **Segmentation** – Handles large messages

Working (Simple)

1. Message is hashed.
2. Hash is encrypted with sender's private key (digital signature).
3. Message is compressed.
4. Compressed message is encrypted using a session key.

5. Session key is encrypted with receiver's public key.
6. Receiver reverses all steps.

PGP uses:

- RSA / DSS for keys
- IDEA / AES for encryption
- SHA for hashing

Comparison (Exam)

S/MIME	PGP
Uses X.509 certificates	Uses web of trust
Standardized	Software-based
Used in enterprises	Used by individuals
CA based trust	User based trust

One-Line Exam Answer

S/MIME and PGP are e-mail security mechanisms that provide confidentiality, authentication, integrity, and non-repudiation using encryption, digital signatures, and public key cryptography.